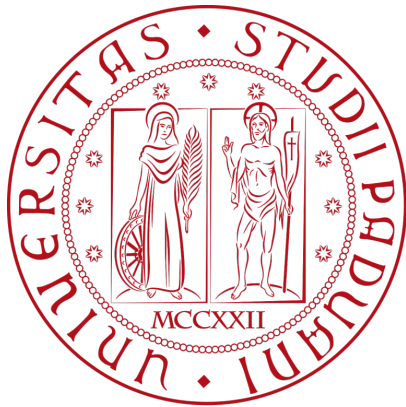


dictionary



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Glossario

Stato	In progress
Versione	0.5
Autori	Davide Lorenzon, Ana Maria Draghici, Filippo Guerra
Verificatori	-
Uso	Interno
Destinatari	Esterni ed interni

Vers.	Data	Autore	Verificatore	Descrizione
0.5	2026-02-26	Ana Maria Draghici	-	Aggiornamento del glossario con nuovi termini e abbreviazioni
0.4	2025-12-16	Filippo Guerra	Ana Maria Draghici	Aggiornamento del glossario con tutti i termini di dominio presenti nei documenti del progetto forniti dall'azienda BlueWind. Aggiunta della sezione Abbreviazioni all'interno del documento.
0.3	2025-11-16	Ana Maria Draghici	Ana Maria Draghici	Aggiunti i termini: Decision tree, Dashboard, CSV, XML, JSON, PDF, Dispositivo radio, Importazione, Interfaccia, Norma armonizzata, Pass, Fail, Not Applicable (N.A.), Requisito funzionale, Requisito non funzionale, Stakeholder, Manutenzione, Editor grafico, Wi-Fi, LTE, BT, IoT.
0.2	2025-11-15	Ana Maria Draghici	Ana Maria Draghici	Aggiunti i termini: Attore, Caso d'uso, Scenario principale, Scenario secondario, Ciclo di vita del progetto, Conformità, Valutazione di conformità, Protezione della rete, Protezione dei dati personali, Prevenzione delle frodi (EN 18031), RED (2014/53/UE), Automated EN18031 Compliance Verification, cybersecurity, report, applicazione desktop, soluzione web-based, UML
0.1	2025-11-09	Davide Lorenzon	Ana Maria Draghici	Stesura iniziale e scripting per l'ordinamento

Indice

1) Introduzione	1
2) Abbreviazioni	2
3) A	3
4) B	4
5) C	5
6) D	6
7) E	7
8) G	8
9) I	9
10) L	10
11) M	11
12) P	12
13) R	13
14) S	14
15) T	15
16) U	16
17) V	17
18) W	18

1) Introduzione

In questo documento vengono raccolti e definiti i termini chiave utilizzati nelle attività di progetto, per garantire chiarezza e coerenza tra i membri del team e tra soggetti esterni coinvolti, come revisori, stakeholder o utenti finali. Lo scopo del glossario è fornire un riferimento unico per abbreviazioni, concetti tecnici e termini normativi utilizzati durante lo sviluppo del progetto, facilitando comunicazione, collaborazione e documentazione.

Per approfondimenti e riferimenti al capitolato e ad altri documenti di progetto, consultare:

- Capitolo del capitolato: [C1](#)
- Documentazione tecnica aggiuntiva: [Repo Progetto](#)

2) Abbreviazioni

Qui sotto si riportano le sigle e abbreviazioni che sono state utilizzate nei documenti di progetto.

AdR	-> Analisi dei Requisiti
AC	-> Actual Cost
BAC	-> Budget at Completion
CPI	-> Cost Performance Index
DoD	-> Definition of Done
EV	-> Earned Value
EAC	-> Estimate at Completion
ETC	-> Estimate to Complete
MU	-> Manuale Utente
MPC	-> Metrica di Qualità del Processo
MPD	-> Metrica di Qualità del Prodotto
NdP	-> Norme di Progetto
PdP	-> Piano di Progetto
PdQ	-> Piano di Qualifica
PBI	-> Product Backlog Item
PoC	-> Proof of Concept
PB	-> Product Baseline
PV	-> Planned Value
RTB	-> Requirement and Technology Baseline
ROF	-> Requisito Obbligatorio Funzionale
RDF	-> Requisito Desiderabile Funzionale
ROQ	-> Requisito Obbligatorio di Qualità
ROV	-> Requisito Obbligatorio di Vincolo
RSI	-> Requirements Stability Index
SPI	-> Schedule Performance Index
TCPI	-> To Complete Performance Index

3) A

3.1) Analisi dei Requisiti

Processo strutturato di raccolta, analisi, classificazione e documentazione dei requisiti. Include interviste, casi d'uso e modellazione, con l'obiettivo di definire con precisione cosa il sistema deve fare prima di iniziare lo sviluppo.

3.2) Attore

Entità esterna (persona fisica, ruolo organizzativo o sistema esterno) che interagisce con il software in almeno un caso d'uso. Non fa parte del sistema, ma vi si interfaccia per raggiungere un obiettivo.

3.3) Actual Cost

Costo reale sostenuto per completare il lavoro svolto fino a un determinato momento, indipendentemente da quanto era stato pianificato o dal valore prodotto.

4) B

4.1) Backlog

Lista ordinata di tutte le attività pianificate e non ancora avviate. Viene aggiornato continuamente durante il progetto e costituisce la fonte da cui si attingono i task per ogni sprint.

4.2) Branch

Ramificazione indipendente del repository Git usata per isolare lo sviluppo. Nel progetto si usano branch distinti per produzione (Main), sviluppo integrato (Develop) e singole funzionalità (Feature).

4.3) BearType

Libreria Python che esegue il controllo dei tipi a runtime, verificando che i valori passati alle funzioni rispettino le annotazioni dichiarate durante l'effettiva esecuzione del programma.

5) C

5.1) Caso d'uso

Descrizione formale di un'interazione tra uno o più attori e il sistema, finalizzata al raggiungimento di un obiettivo specifico. Include precondizioni, scenario principale, scenari alternativi e postcondizioni.

5.2) Ciclo di vita del progetto

L'insieme ordinato delle fasi che scandiscono l'intero sviluppo di un prodotto software: dall'analisi iniziale alla progettazione, implementazione, test, rilascio e manutenzione.

5.3) Ciclo PDCA

Modello iterativo di miglioramento continuo articolato in quattro fasi: Plan (pianifica gli obiettivi), Do (esegui le attività), Check (verifica i risultati ottenuti), Act (applica le correzioni e riparte dal piano).

5.4) Cost Performance Index

Indice di efficienza economica calcolato come rapporto EV/AC . Un valore maggiore di 1 indica che si sta producendo più valore di quanto si stia spendendo.

5.5) Code Smells

Caratteristiche del codice sorgente che, pur non causando errori diretti, indicano possibili problemi strutturali o di manutenibilità e suggeriscono la necessità di un refactoring.

5.6) Cyclomatic Complexity

Metrica che misura la complessità logica di un modulo software contando il numero di percorsi indipendenti nel flusso di controllo. Valori elevati indicano codice difficile da testare e mantenere.

5.7) Coefficient of Coupling

Misura del grado di interdipendenza tra moduli software. Un accoppiamento elevato rende il sistema più rigido e difficile da modificare, testare o riutilizzare in modo indipendente.

6) D

6.1) Definition of Done

Insieme di criteri verificabili e condivisi dal team che determinano quando un'attività può essere considerata formalmente completata, evitando ambiguità sul concetto di finito.

6.2) Docker

Piattaforma di containerizzazione che consente di creare ambienti di esecuzione isolati e riproducibili, garantendo che il software si comporti in modo identico su qualsiasi macchina o sistema operativo.

7) E

7.1) End-user

L'utilizzatore finale del prodotto software, ovvero la persona che interagisce direttamente con il sistema nel contesto reale d'uso, distinta dal committente o dallo sviluppatore.

7.2) Earned Value

Valore economico del lavoro effettivamente completato in un dato momento, espresso in termini di budget pianificato. Usato per misurare l'avanzamento reale del progetto.

7.3) Estimate at Completion

Stima aggiornata del costo totale del progetto al suo completamento, ricalcolata tenendo conto delle performance attuali e dei costi già sostenuti.

7.4) Estimate to Complete

Stima dei costi ancora necessari per completare il lavoro rimanente del progetto, calcolata a partire dallo stato attuale di avanzamento.

8) G

8.1) GitHub Actions

Sistema di integrazione e distribuzione continua (CI/CD) integrato in GitHub, utilizzato per automatizzare build, esecuzione di test e pubblicazione dei documenti ad ogni modifica del repository.

9) I

9.1) Issue

Unità atomica di lavoro tracciata nel sistema di versionamento. Può rappresentare un'attività, un bug, una feature o una modifica documentale, con stato, assegnatario e priorità associati.

9.2) Issue Tracking System

Strumento digitale (come GitHub Issues) usato per pianificare, assegnare, monitorare e storicizzare le attività del progetto, mantenendo traccia dello stato di avanzamento di ciascuna issue.

9.3) Indice di Gulpease

Metrica italiana di leggibilità del testo che valuta la comprensibilità di un documento in base alla lunghezza media delle parole e delle frasi. Valori più alti indicano testi più leggibili.

9.4) Instability Index

Indice compreso tra 0 e 1 che misura la stabilità di un modulo software in base al rapporto tra dipendenze in uscita e totale delle dipendenze. Valori vicini a 1 indicano alta instabilità.

10) L

10.1) Layered Architecture

Architettura software che organizza il sistema in livelli funzionali distinti e sovrapposti (es. presentazione, logica applicativa, accesso ai dati), dove ogni livello interagisce solo con quello adiacente.

11) M

11.1) Milestone

Punto di controllo significativo nella pianificazione del progetto, che segna il completamento di una fase o il raggiungimento di un obiettivo intermedio rilevante, come RTB o Product Baseline.

11.2) Merge

Operazione che unisce il codice di un branch secondario approvato nel branch di destinazione, rendendo effettive le modifiche nel progetto principale.

11.3) MyPy

Strumento di analisi statica per Python che verifica la correttezza dei tipi dichiarati nel codice senza eseguirlo, rilevando potenziali errori in fase di sviluppo.

11.4) MVC

Modello architetturale che separa un'applicazione in tre componenti: Model (dati e logica di business), View (interfaccia utente) e Controller (gestione delle interazioni tra i due).

12) P

12.1) Piano di Qualifica

Documento ufficiale che descrive le strategie, i criteri, le metriche e gli strumenti adottati dal team per svolgere le attività di verifica e validazione durante tutto il ciclo di vita del progetto.

12.2) Pull Request

Richiesta formale di integrare le modifiche sviluppate su un branch secondario nel branch principale. Prevede revisione e approvazione da parte di uno o più membri del team prima che il merge venga eseguito.

12.3) Proof of Concept

Prototipo sperimentale sviluppato per validare la fattibilità di una tecnologia, un'integrazione o un approccio architetturale, prima di procedere con lo sviluppo completo del sistema.

12.4) Product Baseline

Milestone che segna il completamento della progettazione architetturale e della codifica del prodotto, includendo test, documentazione tecnica e tutto il necessario per il rilascio finale.

12.5) Poetry

Strumento per la gestione delle dipendenze e degli ambienti virtuali in Python, che semplifica la dichiarazione, l'installazione e la risoluzione dei pacchetti necessari al progetto.

12.6) Planned Value

Valore economico del lavoro che avrebbe dovuto essere completato entro un dato momento secondo la pianificazione iniziale. Costituisce il riferimento temporale del progetto.

13) R

13.1) Requisito

Rappresenta un'esigenza che il sistema deve soddisfare. Dal lato utente, è ciò di cui ha bisogno per raggiungere un obiettivo; dal lato tecnico, è una capacità che il sistema deve implementare per rispondere a tale esigenza. Può essere funzionale (cosa fa il sistema) o non funzionale (come lo fa).

13.2) Retrospettiva di Sprint

Riunione che si svolge al termine di ogni sprint. Il team analizza cosa ha funzionato, cosa ha creato problemi e quali azioni intraprendere per migliorare nel ciclo successivo.

13.3) Requirement and Technology Baseline

Prima milestone formale del progetto, che comprende il completamento dell'Analisi dei Requisiti, la definizione delle tecnologie adottate e le attività di prototipazione (PoC).

13.4) Responsabile Tecnico

Ruolo con permessi avanzati all'interno del sistema, abilitato alla gestione, modifica e supervisione dei decision tree e delle configurazioni di sistema non accessibili agli utenti standard.

13.5) Ruff

Lint e formater per codice Python ad alte prestazioni, utilizzato per rilevare errori stilistici, violazioni di convenzioni e per formattare automaticamente il codice in modo uniforme.

13.6) Requirements Stability Index

Metrica che misura quanto i requisiti rimangono stabili nel tempo, calcolando il rapporto tra requisiti modificati o eliminati e il totale dei requisiti definiti. Un valore alto indica buona stabilità.

14) S

14.1) Scenario principale

La sequenza di passi ideale di un caso d'uso, quella che si verifica quando tutto va come previsto, senza errori o deviazioni dal flusso atteso.

14.2) Scenario secondario

Sequenza alternativa che si attiva in caso di eccezioni, errori o condizioni particolari rispetto allo scenario principale. Definisce il comportamento del sistema nei casi non ordinari.

14.3) Sprint

Iterazione di sviluppo a durata fissa (nel progetto, bisettimanale) al termine della quale si produce un incremento verificabile del prodotto. È l'unità base del metodo Scrum.

14.4) Schedule Performance Index

Indice di rispetto delle tempistiche calcolato come rapporto EV/PV. Un valore maggiore di 1 indica che il progetto è in anticipo rispetto alla pianificazione.

14.5) Statement Coverage

Metrica di copertura del codice che indica la percentuale di istruzioni eseguite durante i test automatici. Un valore alto riduce la probabilità che comportamenti non testati nascondano difetti.

15) T

15.1) Tracciamento automatico

Meccanismo che collega sistematicamente i casi d'uso ai requisiti corrispondenti tramite script dedicati, garantendo coerenza e completezza tra le specifiche e la documentazione prodotta.

15.2) Typst

Linguaggio di markup moderno utilizzato per la composizione tipografica dei documenti del progetto, pensato come alternativa a LaTeX con sintassi più semplice e compilazione più rapida.

15.3) TurboScribe AI

Strumento basato su intelligenza artificiale utilizzato per la trascrizione automatica delle riunioni, producendo verbali testuali a partire da registrazioni audio o video.

15.4) Time Efficiency

Metrica che misura l'efficienza temporale del team, calcolata come rapporto tra il tempo stimato per un'attività e il tempo effettivamente impiegato per completarla.

15.5) To Complete Performance Index

Indice che indica il livello di efficienza economica necessario per completare il progetto rispettando il budget residuo disponibile.

16) U

16.1) UML

Linguaggio di modellazione standardizzato (Unified Modeling Language) usato per rappresentare visualmente strutture, comportamenti e interazioni di un sistema software tramite diagrammi come casi d'uso, classi, sequenze e altri.

17) V

17.1) Verifica

Processo di controllo interno che assicura che il prodotto sia stato costruito correttamente rispetto alle specifiche definite. Risponde alla domanda: Stiamo costruendo il sistema nel modo giusto?

17.2) Validazione

Processo che assicura che il prodotto costruito corrisponda alle reali esigenze dell'utente finale. Risponde alla domanda: Stiamo costruendo il sistema giusto?

18) W

18.1) Workflow documentale

Modello a stati che governa il ciclo di vita di ogni documento: Backlog → In lavorazione → In verifica → In validazione → Done, garantendo un processo controllato e tracciabile per ogni prodotto documentale.